

Fundamentos de Redes de Computadores

Projeto da Disciplina - 2026.1

Temática: AuctionCIn - Sistema de Leilão Multi-Usuário

!!O que está em vermelho não é negociável!!

Neste projeto, a equipe desenvolverá um sistema de leilão onde múltiplos clientes podem disputar itens em tempo real. O projeto foca na implementação de confiabilidade sobre UDP e na gerência de estado no servidor.

Serão postadas atividades no Google Classroom referentes a cada etapa do projeto. A equipe deve realizar **todas** as entregas para que a nota final (soma das 3 etapas) seja validada. Em cada etapa, deverá ser entregue através do Google Classroom uma pasta compactada com o repositório do projeto ou um link do github com uma pasta para cada entrega. A atividade deverá ser entregue **por cada um dos membros da equipe**, nesses termos.

- **Obs:** Cada entrega deve conter, em sua pasta, um README.md exclusivo daquela entrega, com instruções de execução e eventuais observações necessárias e o nome dos integrantes. **Comentem o código!**

A seguir, serão descritas as especificações e requisitos de cada uma das etapas:

1. Primeira Etapa: Transmissão de arquivos com UDP

Implementação de comunicação **UDP** utilizando a biblioteca **socket** na linguagem **Python**, com envio e devolução de arquivo (o arquivo deve ser enviado pelo cliente, armazenado no servidor e devolvido ao cliente), em pacotes de até 1024 bytes (**buffer_size**). Não é necessária a implementação de transferência confiável nessa etapa, o servidor armazena, renomeia (ex: adicionando um prefixo "leilao_") e devolve os arquivos renomeados ao cliente para confirmar o recebimento. **Prazo para primeira entrega: 28/04**

A implementação deverá ser realizada conforme os requisitos a seguir:

- Nessa entrega, deve ser implementado o envio e devolução de **arquivos**, reforçando que o envio de strings não é suficiente. Sugerimos que testem o programa para **ao menos dois tipos** de arquivos, como por exemplo um .txt e uma imagem.
- É necessário a alteração do nome do arquivo antes da devolução ao cliente para demonstrar o funcionamento correto do código.

- É recomendável a divisão do cliente e servidor em arquivos (.py) diferentes, para a execução de funcionalidades específicas. Lembrar que essa modularização é importante dado a escalabilidade das próximas entregas.
- **Dica:** É uma dúvida comum se mensagem é diferente de arquivo, ou se ambos podem ser enviados em um só pacote com `buffer_size` infinito. Nesse contexto, uma mensagem ou um arquivo são e devem ser considerados a mesma coisa, bits, e devem seguir o mesmo fluxo. O que muda é que arquivos ou mensagens maiores que o **buffer_size** (1024 bytes) devem ser fragmentados em pacotes e reconstruídos no receptor. Recomendamos utilizar `file.read` e `file.write` (Python).

2. Segunda Etapa: Implementando uma transferência confiável com RDT 3.0

Utilizando-se do código feito na etapa anterior, os grupos devem agora simular uma transferência confiável segundo o canal de transmissão **RDT3.0**, apresentado na disciplina e presente no Kurose. **Prazo para segunda entrega: 26/05**

A implementação deverá ser realizada conforme os requisitos a seguir:

- Nessa etapa, seu programa ainda deve realizar o **envio, armazenamento e devolução de arquivos de tipos diferentes**
- Cada passo do seu programa **deve ser apresentado na linha de comando**, de modo a se ter compreensão do que está acontecendo
- O socket UDP de cada cliente e do servidor deverá contar com **transmissão confiável**, implementada em camada de aplicação, segundo o **RDT 3.0** que consta no livro “Redes de Computadores e a Internet” do Kurose.
- Deve ser implementado um **gerador de perdas de pacotes aleatórios**, levando ao timeout no recebimento desses pacotes, de modo a testar a confiabilidade do canal e a eficiência do RDT3.0

Obs.: O RDT3.0 apresentado pelo Kurose utiliza um **checksum**. Entretanto, para esse projeto, **não é necessária** a implementação do checksum, uma vez que já existem checksums no protocolo UDP e na camada de enlace.

3. Terceira Etapa: O Sistema AuctionCin com paradigma cliente-servidor

O CIn desenvolveu um sistema de leilão chamado **AuctionCin** para promover a atividade de leilões por todo território nacional. O projeto consiste na implementação de um sistema de leilões online multiusuário, exibido em linha de comando. O Sistema reutiliza as etapas anteriores, mas nesta entrega o foco é apenas a aplicação final, sem exibir históricos de envio/devolução de arquivos. O uso do RDT 3.0 deve ser mantido.

Prazo para terceira entrega: 26/06

A implementação deverá ser realizada conforme os requisitos a seguir:

- É necessário que o **AuctionCin** funcione **para ao menos dois clientes simultaneamente**, ou seja, deverão ser abertos o terminal do servidor e ao menos dois terminais de clientes (com portas diferentes) sem que ocorra interrupção do funcionamento.
- O socket UDP de cada cliente e do servidor deverá contar com **transmissão confiável**, implementada em camada de aplicação segundo o **RDT 3.0** que consta no livro “Redes de Computadores e a Internet” do Kurose.
- A equipe deverá explicar o código e mostrar o chat funcionando através de um vídeo de, no máximo, 15 minutos. Todos os integrantes do grupo devem participar.

As funcionalidades serão executadas/solicitadas através de linhas de comando pelo cliente e serão interpretadas pelo sistema.

Funcionalidade	Comando
Conectar ao sistema	login <nome_do_usuario>
Dar um Lance	bid <id_item> <valor>
Ver itens e preços	list
Ver quem está ganhando	status
Sair do sistema	logout

Requisitos das funcionalidades:

- Dois usuários não podem ter o mesmo nome

- Cada item precisa ter um <id_item> para que o usuário possa dar lances facilmente
- Cada item que vai a leilão pode ser pré-definido, e pode ser um arquivo .txt (por exemplo: Carro.txt), utilizado apenas para referenciar que é um carro.
- Quando um usuário dá um lance através do comando bid <id_item> <valor>, todos os usuários conectados recebem essa mensagem em atualização simultânea.
- O valor do lance precisa ser maior que o lance anterior (recomendado usar float, caso o usuário queira um valor apenas 1 centavo mais caro).
- Após o leilão, o item é enviado ao usuário com o lance mais alto.
- Cada item tem disponibilidade de tempo, o item fica disponível por 60 segundos ou até 5 lances serem enviados.

Descrições dos comandos:

login <nome_do_usuario>

O cliente passa a participar do leilão quando consegue fazer login passando um nome de usuário. Dois usuários não podem se conectar ao sistema utilizando o mesmo nome.

Recomendamos que haja uma lista com informações automáticas com nomes que os associam a um par (ip:porta) já definido. De toda forma, cada cliente deve ter uma porta única.

Ao realizar o login, o servidor deve enviar a seguinte mensagem para o cliente: “*você está online*”.

Cliente deve estar online para acessar as demais funcionalidades do sistema

logout

Ao usar o comando logout, o cliente deve ser excluído da lista de usuários online do servidor.

Ao sair, o cliente só consegue acessar as demais funcionalidades realizando o login novamente.

Exemplo de lista de contatos:

Nome	IP:PORTA
Felipe Maltez	192.168.100.100:5000
Vitor Azevedo	192.168.100.100:5500

- **Obs.:** Considere clientes como processos. Verifique que cada cliente tem portas diferentes. Pode considerar o IP local como padrão para todos os clientes.